

See Sparsity Happen. Not Just Hear About It.

Inside BDH turns one line of model code into a live learning experiment: change the input, watch the latent state change, and verify exactly what ReLU does inside a trained toy Dragon Hatchling model.

Public artifact	https://inside-bdh-1.onrender.com
Source repository	https://github.com/himani27301/inside-bdh
Central topic	Sparse non-negative activations + state interpretability

Why this project exists

Frontier-model papers often compress the interesting part into a sentence such as "activations are sparse and positive." That is easy to repeat and surprisingly hard to internalize. What is sparse? What was there before the zeros appeared? Does the pattern actually change when the input changes? And how can a learner tell whether a visualization is a real model state or merely an animation?

Our falsifiable claim: BDH converts a signed latent projection into an input-dependent, non-negative sparse state; changing the input changes which latent units survive.

The goal is not to explain all of BDH. It is to make one mechanism visible enough that a learner can predict it, test it, and challenge it.

The exact computation we instrument

The public toy BDH implementation contains a compact sequence of operations. We expose the two states around its ReLU transformation:

```
x_latent = x @ self.encoder
x_sparse = ReLU(x_latent)
```

x_latent is a signed projection: entries can be negative or positive. ReLU maps every value at or below zero to zero and leaves positive values unchanged. Therefore **x_sparse** is non-negative, and its zero pattern is directly checkable against the pre-ReLU state. The amount of activity is still learned and input-dependent; ReLU is not a fixed "5% sparsity knob."

Pre-ReLU x_latent	ReLU rule	Post-ReLU x_sparse
-1.20, 0.80, -0.30, 2.10	$\max(0, x)$	0.00, 0.80, 0.00, 2.10

The BDH paper reports sparse, positive activation vectors and broader interpretability properties.[1] Our demo only claims what we can show with the toy checkpoint: the live transformation, its activity statistics, and how those statistics change with input.

What the learner can actually change

The interface runs two inputs through the *same* checkpoint. This isolates the input as the main independent variable. Learners can compare natural language, repetition, technical text, or noise; select Layers 1 through 6; inspect x/y/xy activity; inspect strongest last-token units; and see top-unit overlap.

Layer-1 input	x_sparse active	Relative
Natural sentence	17.95%	
Repetition	20.29%	
Noise-like	42.52%	
Technical sentence	17.68%	

For the natural sentence, **82.05%** of Layer-1 pre-ReLU values are non-positive and **82.05%** of post-ReLU values are exactly zero. Top-20 last-token unit overlap with the natural sentence is 20% for repetition, 0% for noise, and 100% for the second natural-language sentence in this small test.

These are deliberately labeled local measurements from our independently trained Tiny-Shakespeare toy checkpoint. They are not general BDH guarantees or a reproduction of Pathway's larger-model results.

The 60-second learning loop

1) Start with natural vs repetition. 2) Inspect Layer 1. 3) Verify **Before ReLU -> after ReLU** by comparing expected and observed zeroing. 4) Switch Input B to noise and rerun. 5) Predict the ReLU output in the mini-quiz. This moves the learner from explanation to prediction to live verification.

How this fits the interpretability landscape

Sparse internal representations are also central to recent sparse-autoencoder (SAE) work. Cunningham et al. use SAEs to decompose language-model activations into sparsely activating, more interpretable features.[2] Gao et al. scale sparse autoencoders to millions of latents and study reconstruction-sparsity trade-offs and feature-quality metrics.[3] The mechanism is different: an SAE is typically a post-hoc learned dictionary over another model's activations, while Inside BDH reads sparse latent activity directly from the toy BDH forward pass.

BDH-CQ is a later member of the same architecture family focused on in-context learning with recurrent latent reasoning.[4] This demo does not run BDH-CQ.

What we refuse to overclaim

- **This is a toy checkpoint.** It is a byte-level model trained on Tiny Shakespeare from Pathway's public toy implementation, not an official production checkpoint.
- **Our activity percentages are local measurements.** They describe this checkpoint and these inputs; they are not a reproduction of published large-model sparsity rates.
- **ReLU is only part of the story.** It explains non-negativity and zeroing, but not every learned sparsity, connectivity, memory, or semantic property reported for larger BDH systems.
- **Top-unit overlap is not semantic proof.** It is a lightweight comparison metric for the teaching interaction.
- **SAE research is context, not evidence that BDH neurons equal SAE features.** The mechanisms and learned objects differ.

Why the project is still useful

The most valuable outcome is not a benchmark number. It is technical ownership. A learner can trace a displayed quantity to a tensor, a tensor to a line of code, and a line of code to a prediction they can test. That is the level at which frontier architectures become teachable.

The same substrate can later support richer checkpoints, concept-conditioned activation analysis, longer-sequence state evolution, or side-by-side Transformer baselines. Those extensions should keep the same principle: **expose the state, make the claim falsifiable, and label the evidence honestly.**

Primary sources

- [1] Kosowski, A. et al. (2025). The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain. arXiv:2509.26507.
- [2] Cunningham, H. et al. (2023). Sparse Autoencoders Find Highly Interpretable Features in Language Models. arXiv:2309.08600.
- [3] Gao, L. et al. (2024). Scaling and Evaluating Sparse Autoencoders. arXiv:2406.04093.
- [4] Engdahl, B. et al. (2026). BDH-CQ: In-Context Learning with Recurrent Latent Reasoning. arXiv:2608.09888.

Public artifact: <https://inside-bdh-1.onrender.com>

Source repository: <https://github.com/himani27301/inside-bdh>